Start with a one-dimensional quadratic, then a two-dimensional example. Write the clean and corrupted update equations for gradient descent and momentum. Derive how a single error propagates. Then study Adam's equations numerically before attempting more general theory.

One dimensional quadratic propagation

Let us take a simple one dimensional equation:

$$f(x) = x^2.$$

The gradient for this equation is

$$f'(x) = 2x$$

and gradient descent for this equation holds the form

$$x_{t+1} = x_t - 2\alpha x_t$$

The update vector will be $v_t = \gamma v_{t-1} + \alpha \nabla_\theta J(\theta)$ to test momentum.
Let us follow 5 update equations for a clean run starting from $\alpha = 0.1, \gamma = 0.9$:

$$g_0 = 2x_0, v_1 = 0.2x_0, x_1 = 0.8x_0$$

$$g_1 = 2x_1 = 1.6x_0, v_2 = 0.18x_0 + 0.16x_0 = 0.34x_0, x_2 = 0.8x_0 - 0.34x_0 = 0.46x_0$$

$$g_2 = 2x_2 = 0.92x_0, v_3 = 0.306x_0 + 0.092x_0 = 0.398x_0, x_3 = 0.46x_0 - 0.398x_0 = 0.062x_0$$

$$g_3 = 2x_3 = 0.124x_0, v_4 = 0.3582x_0 + 0.0124x_0 = 0.3706x_0, x_4 = 0.062x_0 - 0.3706x_0 = -0.3086x_0$$

$$g_4 = 2x_4 = -0.6172x_0, v_5 = 0.3335x_0 - 0.06172x_0 = 0.2717x_0, x_5 = -0.3086x_0 - 0.2717x_0 = -0.5803x_0$$

Now, let's say that on time step 2, a value of δ was added to the gradient:

$$\hat{g_2} = 0.92x_0 + \delta, \hat{v}_3 = 0.398x_0 + 0.1\delta, \hat{x}_3 = 0.062x_0 - 0.1\delta$$

$$\hat{g}_3 = 0.124x_0 - 0.2\delta, \hat{v}_4 = 0.3706x_0 + 0.07\delta, \hat{x}_4 = -0.3086x_0 - 0.17\delta$$

$$\hat{g}_4 = -0.6172x_0 - 0.34\delta, \hat{v}_5 = 0.2717x_0 + 0.029\delta, \hat{x}_5 = -0.5803x_0 - 0.199\delta$$

Depending on the value of δ added, the calculations can be thrown off for a long time and finding the minimum will take longer.

Two dimensional quadratic propagation

Now, let's do the same thing with a two dimensional quadratic:

$$f(x, y) = x^2 + xy + y^2$$

The gradient vector for this equation is:

$$\nabla f(x, y) = \begin{bmatrix} 2x + y \\ x + 2y \end{bmatrix}$$

The update equations are analogous:

$$g_0 = \begin{bmatrix} 2x_0 + y_0 \\ x_0 + 2y_0 \end{bmatrix}, v_1 = \begin{bmatrix} 0.2x_0 + 0.1y_0 \\ 0.1x_0 + 0.2y_0 \end{bmatrix}, \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} 0.8x_0 - 0.1y_0 \\ -0.1x_0 + 0.8y_0 \end{bmatrix}.$$

$$g_1 = \begin{bmatrix} 1.5x_0 + 0.6y_0 \\ 0.6x_0 + 1.5y_0 \end{bmatrix}, v_2 = \begin{bmatrix} 0.33x_0 + 0.15y_0 \\ 0.15x_0 + 0.33y_0 \end{bmatrix}, \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 0.47x_0 - 0.25y_0 \\ -0.25x_0 + 0.47y_0 \end{bmatrix}.$$

$$g_2 = \begin{bmatrix} 0.69x_0 - 0.03y_0 \\ -0.03x_0 + 0.69y_0 \end{bmatrix}, v_3 = \begin{bmatrix} 0.366x_0 + 0.132y_0 \\ 0.132x_0 + 0.366y_0 \end{bmatrix}, \begin{bmatrix} x_3 \\ y_3 \end{bmatrix} = \begin{bmatrix} 0.104x_0 - 0.382y_0 \\ -0.382x_0 + 0.104y_0 \end{bmatrix}.$$

$$g_3 = \begin{bmatrix} -0.174x_0 - 0.66y_0 \\ -0.66x_0 - 0.174y_0 \end{bmatrix}, v_4 = \begin{bmatrix} 0.312x_0 + 0.0528y_0 \\ 0.0528x_0 + 0.312y_0 \end{bmatrix}, \begin{bmatrix} x_4 \\ y_4 \end{bmatrix} = \begin{bmatrix} -0.208x_0 - 0.4348y_0 \\ -0.4348x_0 - 0.208y_0 \end{bmatrix}.$$

$$g_4 = \begin{bmatrix} -0.8508x_0 - 1.0776y_0 \\ -1.0776x_0 - 0.8508y_0 \end{bmatrix}, v_5 = \begin{bmatrix} 0.19572x_0 - 0.06024y_0 \\ -0.06024x_0 + 0.19572y_0 \end{bmatrix}, \begin{bmatrix} x_5 \\ y_5 \end{bmatrix} = \begin{bmatrix} -0.40372x_0 - 0.37456y_0 \\ -0.37456x_0 - 0.40372y_0 \end{bmatrix}.$$

Now, let's say that on time step 2, a value of δ was added to the gradient:

$$\hat{g}_2 = \begin{bmatrix} 0.69x_0 - 0.03y_0 + \delta \\ -0.03x_0 + 0.69y_0 \end{bmatrix}, \hat{v}_3 = \begin{bmatrix} 0.366x_0 + 0.132y_0 + 0.1\delta \\ 0.132x_0 + 0.366y_0 \end{bmatrix}, \begin{bmatrix} \hat{x}_3 \\ \hat{y}_3 \end{bmatrix} = \begin{bmatrix} 0.104x_0 - 0.382y_0 - 0.1\delta \\ -0.382x_0 + 0.104y_0 \end{bmatrix}.$$

$$\hat{g}_3 = \begin{bmatrix} -0.174x_0 - 0.66y_0 - 0.2\delta \\ -0.66x_0 - 0.174y_0 - 0.1\delta \end{bmatrix}, \hat{v}_4 = \begin{bmatrix} 0.312x_0 + 0.0528y_0 + 0.07\delta \\ 0.0528x_0 + 0.312y_0 - 0.01\delta \end{bmatrix}, \begin{bmatrix} \hat{x}_4 \\ \hat{y}_4 \end{bmatrix} = \begin{bmatrix} -0.208x_0 - 0.4348y_0 - 0.17\delta \\ -0.4348x_0 - 0.208y_0 + 0.01\delta \end{bmatrix}.$$

$$\hat{g}_4 = \begin{bmatrix} -0.8508x_0 - 1.0776y_0 - 0.33\delta \\ -1.0776x_0 - 0.8508y_0 - 0.15\delta \end{bmatrix}, \hat{v}_5 = \begin{bmatrix} 0.19572x_0 - 0.06024y_0 + 0.03\delta \\ -0.06024x_0 + 0.19572y_0 - 0.024\delta \end{bmatrix}, \begin{bmatrix} \hat{x}_5 \\ \hat{y}_5 \end{bmatrix} = \begin{bmatrix} -0.40372x_0 - 0.37456y_0 - 0.2\delta \\ -0.37456x_0 - 0.40372y_0 + 0.034\delta \end{bmatrix}.$$

Now, even though we only added the error to x_0, it has propagated through the xy term into the other parameter. This is one of the dangers of these types of errors, it doesn't only impact the magnitude.

Adam formulas

Along with the average of past squared gradients, Adam stores an exponentially decaying average of past gradients (not squared):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

These are estimates of the first and second moment respectively (mean and uncentered variance), but they are biased towards 0 especially when the beta terms (decay terms) are close to 1. Correcting for bias:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

we get the update rule:

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

How is Adam impacted when an error occurs?

Hypothesis:
Intuitively, any error will find its' way into both the first and second moment of Adam. While $\hat{v}_t$ will essentially undergo an L2 regularization, lessening the potential magnitude of the error, $\hat{m}_t$ will remain as is. This means there is potential for the error to continue growing and fluctuating over time. Also, any term that has an error occur and is coupled to another feature (in terms of actual ML, related to it) will spread its' error to all coupled features. In the real world, features are often related to one another and although it is common practice to perform PCA or something similar to get rid of collinearity, this may still impact model performance.

CPU experiments

To test the theory I had and look at the optimizers through more timesteps, I wrote some code.
I ran the 1-dimensional and 2-dimensional momentum and Adam optimizers on the functions mentioned above through 100 timesteps and kept track of the parameter values, the velocity and the first/second moments for momentum and Adam respectively. I experimented with 5 scenarios:

1. Uncorrupted - a base run to see how the model works.
2. Corrupted keep - an error is added into the gradient and no preventative measures are taken.
3. Corrupted reset - an error is added into the gradient, but the optimizer's parameters are reset to 0.
4. Corrupted replace - a hypothetical scenario where an error is added into the gradient and the model somehow recovers the parameters of the optimizer from the uncorrupted run.
5. Uncorrupted reset - an error isn't added, but the parameters are still reset.
   The parameters for the tests are as follows:
   $x_0 = 5, y_0 = 5, \alpha = 0.1, \gamma = 0.9, \beta_1 = 0.9, \beta_2 = 0.99, \delta = 100, t_{corruption} = 50$

1-D Momentum

x oscillated around the optimal value with decreasing amplitude. As expected, the uncorrupted runs converged and the corrupted runs got thrown off by the error with the biggest convergence delay happening in the "corrupted keep" scenario.
A substantially large error can throw off momentum for a long time, possibly multitudes longer than training would take normally.

1-D Adam

The uncorrupted, corrupted keep and corrupted replace parameters stay relatively close to one another, while the reset scenarios get thrown off significantly. This suggests that Adam resists these error injections better when not tampered with at all; in fact, resetting the optimizer (regardless of the error occurring or not) can be more harmful than not doing anything at all. However, replacing the optimizer's moments is still better than keeping the corrupted ones since if the error is sufficient it will continue to propagate and diverge the "keep" trajectory from the clean one.
This also suggests that resetting is not the preferred strategy to deal with error injections. In a false positive scenario, resetting will be much worse than continuing with a faulty optimizer.

In the parameter update graph, the lines for the uncorrupted and replaced graph overlap(the "keep" line looks like it was reset several timesteps into the past).

2-D Momentum

The results are largely the same as from the 1-dimensional example with one minor update: replacing the velocity leads to a smaller parameter amplitude and slightly faster convergence as opposed to resetting it.
This makes sense as:

1. the graph displays the updates of both axes at once and the larger scale allows for closer inspection
2. resetting the optimizer's state here isn't as beneficial because it means that the cumulative velocity vector prior to the injection does not affect the gradient afterwards, leaving only the corrupted weights.

2-D Adam

Identical to the 1-D example, the reset scenarios perform the worst with keeping the optimizer and replacing the weights being the better strategies.

Conclusion

When errors are injected into the gradient, the preferred strategy for both momentum and Adam is to repair the optimizer's weights.
Momentum benefits from resetting the internal state of the optimizer, but Adam does not. This is likely because Adam's second momentum has a stabilizing effect, and given that the error occurs deeper into training it will have less of an effect on convergence.

Revisions

The first version of the deliverable had several algebraic bugs, which have now been fixed. Furthermore, it was impossible to compare optimizer values independently from the weights. The system has been reworked and now the corruption step splits the run into several experiments (4 for momentum, 8 for Adam):

- clean weights, clean optimizer - the base run
- clean weights, corrupted optimizer - leaving the weights from the base run, but leaving the optimizer corrupted
- corrupted weights, clean optimizer - identical to the "replace" run; leaving the optimizer from the base run, but leaving the weights corrupted
- corrupted weights, corrupted optimizer - identical to the "keep" run; everything is corrupted
  Detection delays have also been implemented to see how noticing the error later on would impact runtime and optimal strategies.

Revised conclusion

For 1D momentum on low detection delays, corrupted weights and clean velocities lose amplitude quickly, but on higher detection delays clean weights and corrupted velocities perform better. This trend doesn't follow for the 2D experiment, where no clear optimal can be seen.

An interesting note is that whether or not the the corruption of weights affects Adam depends on the sign of the error. When the error is added to the gradient, the lines for corrupted/clean weights have the same slope even on high detection delays, with the clean hovering closer to the optimum. When an error is subtracted, convergence to the optimum slows down dramatically, leading the clean weight experiments to have lower cumulative excess loss on average. A corrupted m seems to impact convergence far more than a corrupted v on both dimensional experiments in the case of both a positive or negative error. Resetting still proves to be near useless in cases where the optimal is not 0.

Further changes

After more bug fixes and changes to the structure of the code base, I decided to test both the positive and negative error.
The later an error is caught, the more closely the convergence of any model with a corrupted component mirrors a fully corrupted model. Once again, a corrupted m impacts the cumulative loss much more than a corrupted v does. In the case of a late detection time, a case with only a corrupted m did not recover at all on the 2D example.
The sign of the error also affects recovery. In the 2D Adam simulation, a negative error led to, on average, 24% (accounting for the experiments that did not converge in time). This is likely because subtracting a value from the loss, in the most basic terms, makes the model believe that it is closer to the optimum than it actually is. If this propagates into the optimizer parameters, it makes complete sense for the model to then perform worse.